



comp10002/comp20005 program insertionsort.c

Support material for Chapter 7 of
Programming, Problem Solving, and Abstraction with C,
by Alistair Moffat

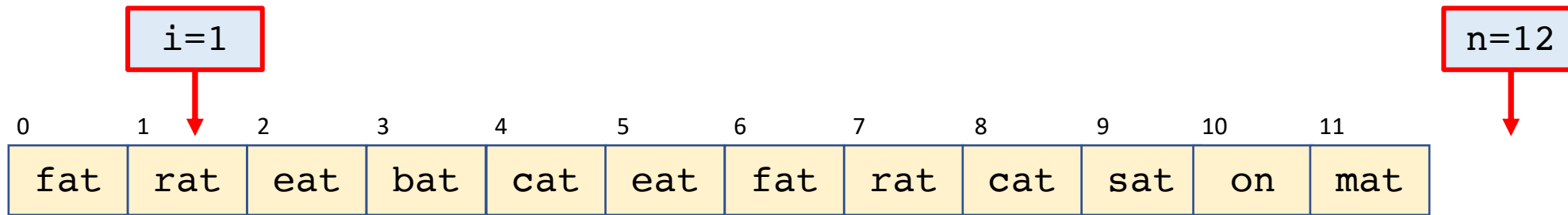
(c) 2023, The University of Melbourne
Prepared by Alistair Moffat, am Moffat@unimelb.edu.au

n=12



0	1	2	3	4	5	6	7	8	9	10	11
fat	rat	eat	bat	cat	eat	fat	rat	cat	sat	on	mat

```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

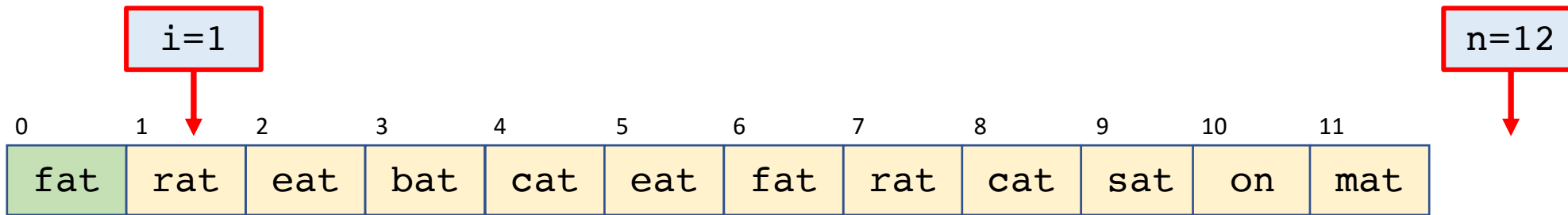


```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

desired **invariant**:
the elements to the left of **i** are sorted...

desired invariant:
the elements to the left of **i** are sorted...

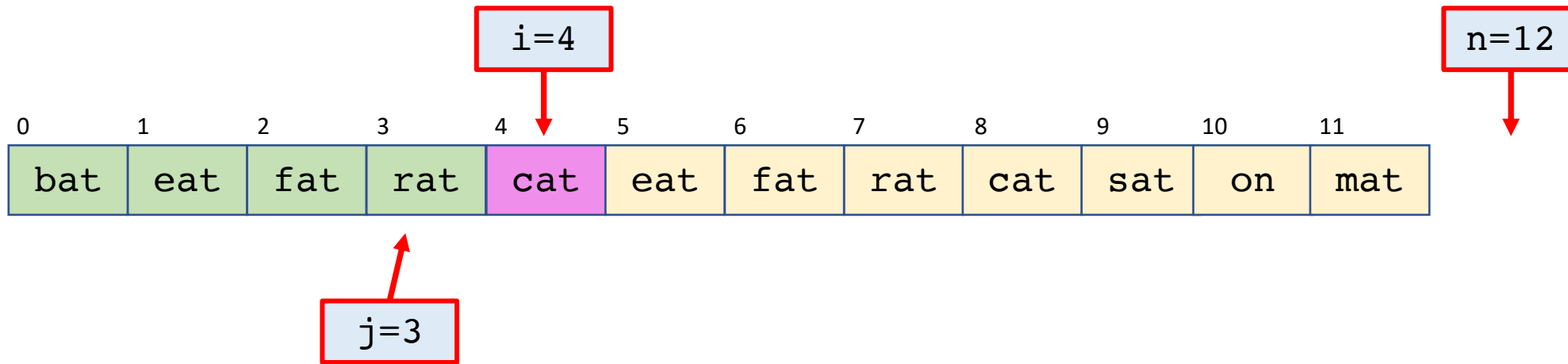
base case:
the assignment **i=1** makes that true



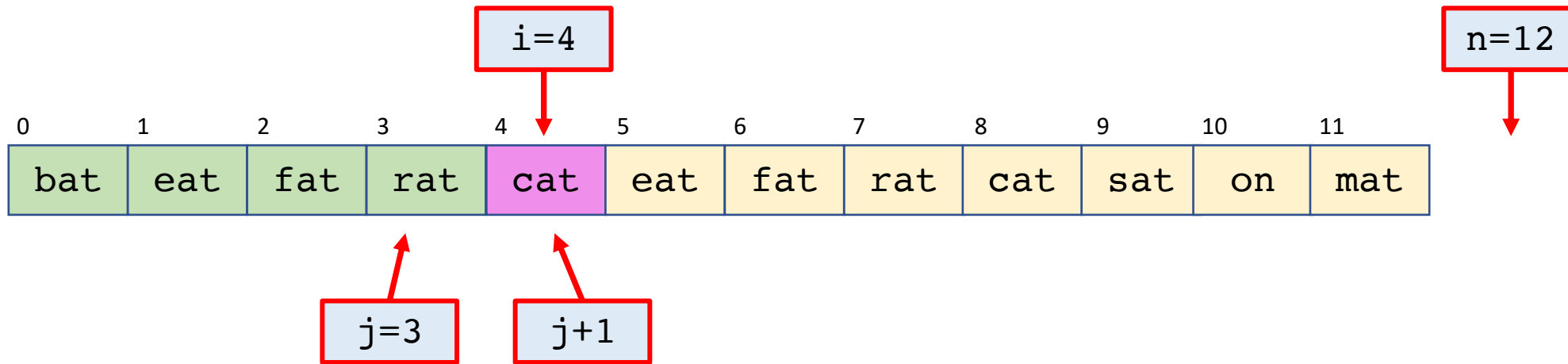
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

desired invariant:
the elements to the left of **i** are sorted...

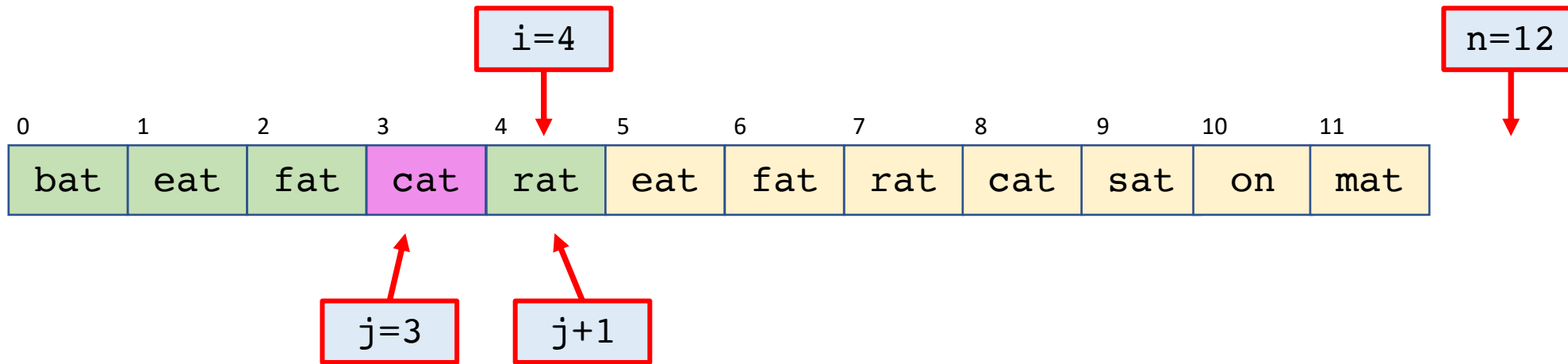
inductive step:
what conditions allow **i** to be incremented?



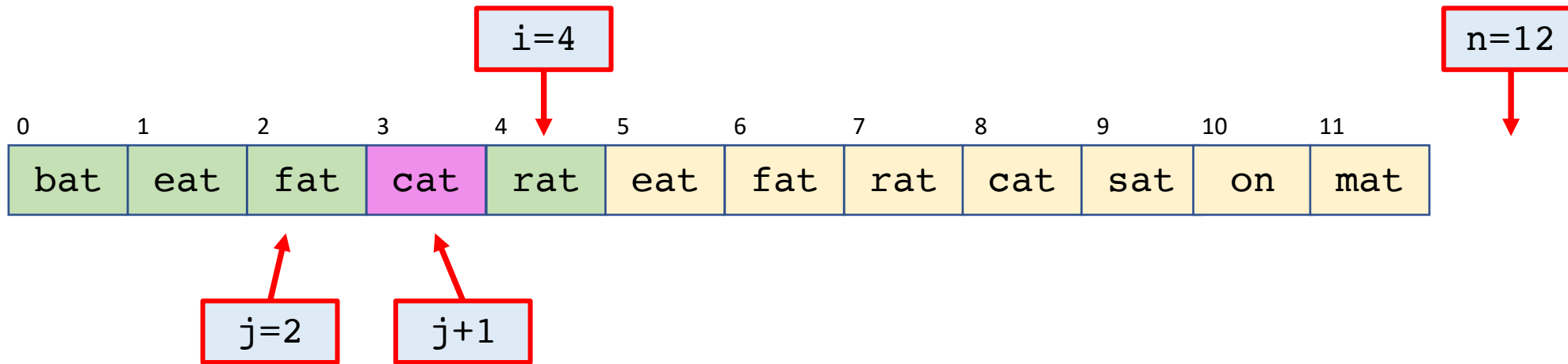
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

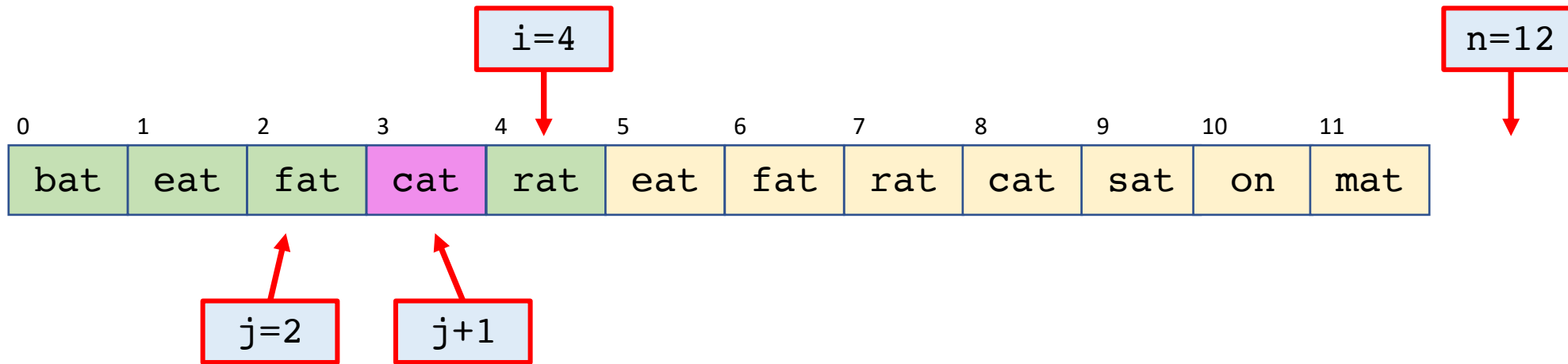
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



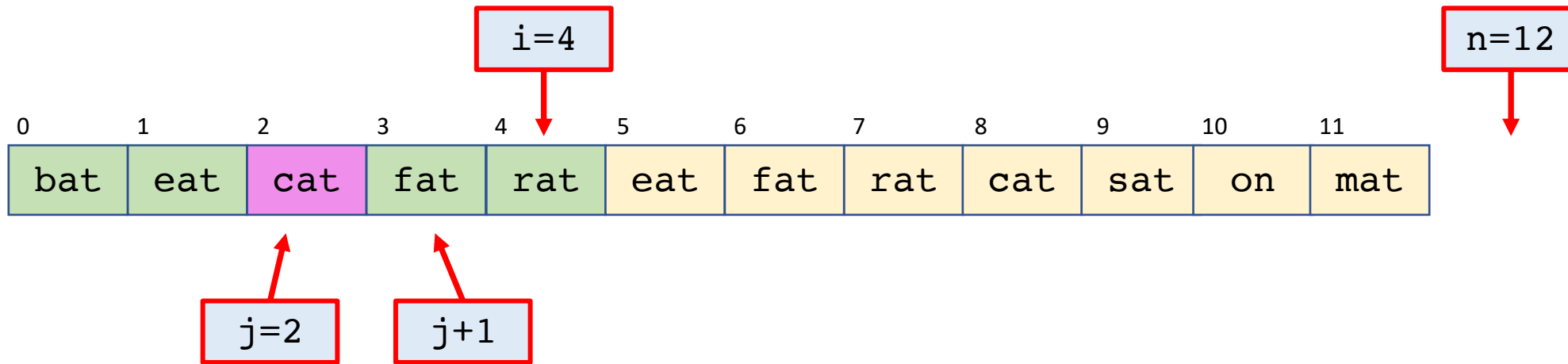
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



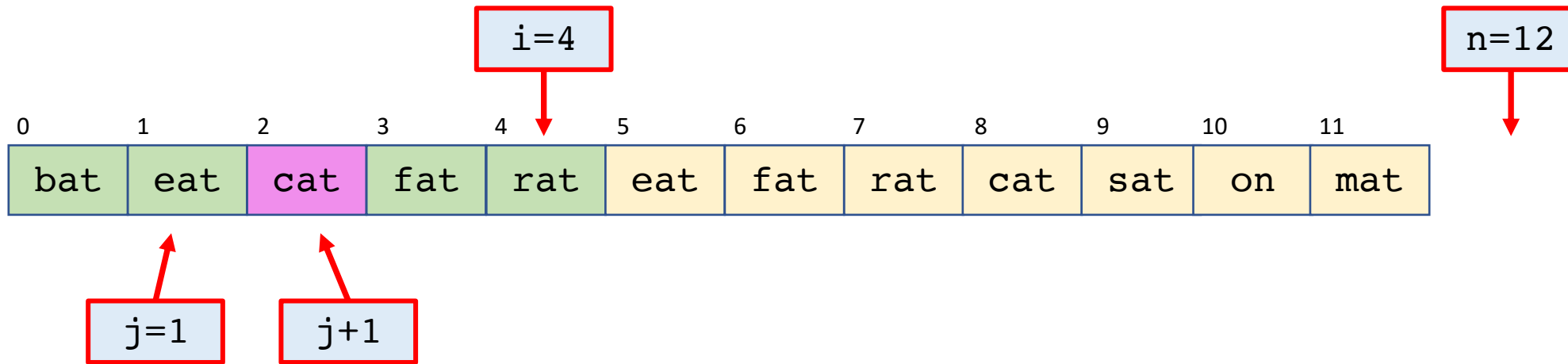
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



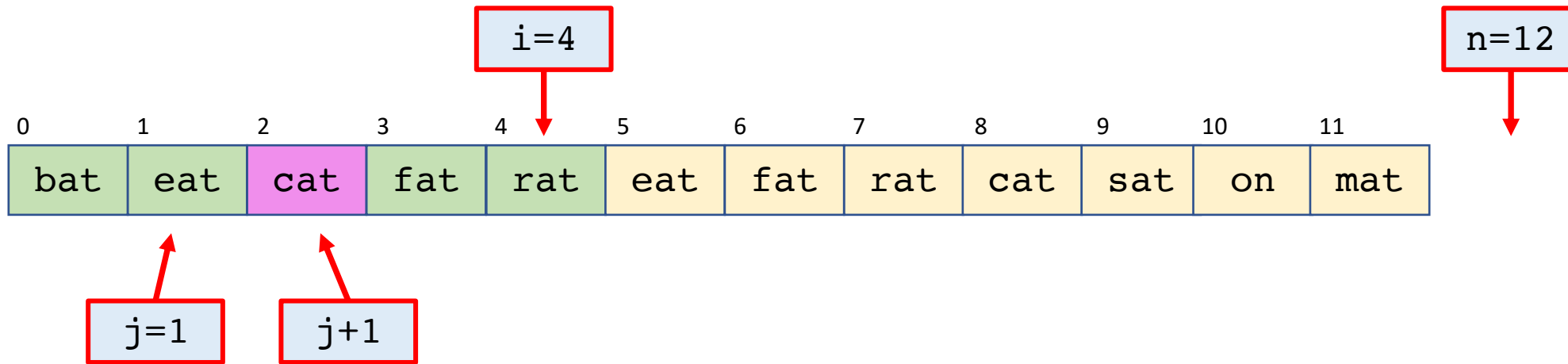
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



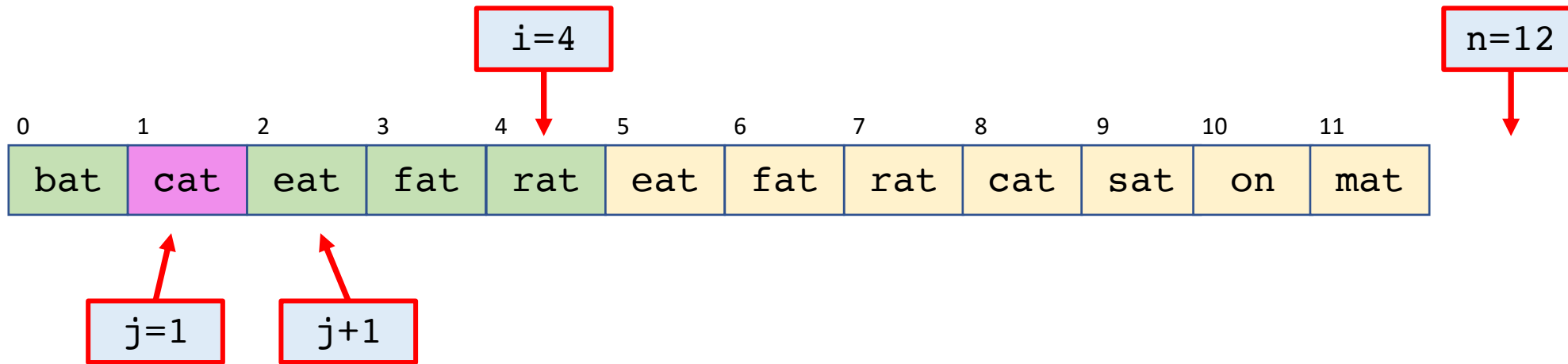
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



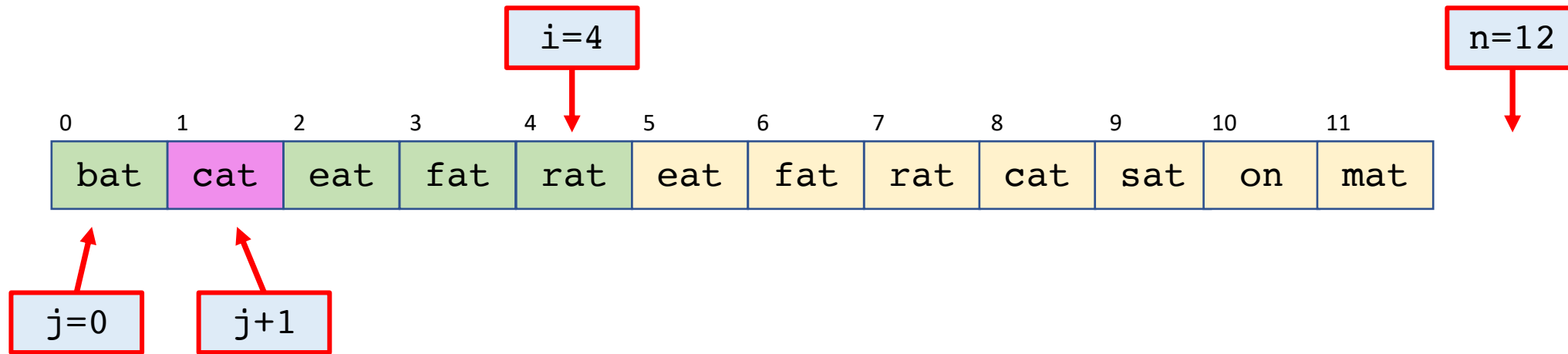
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



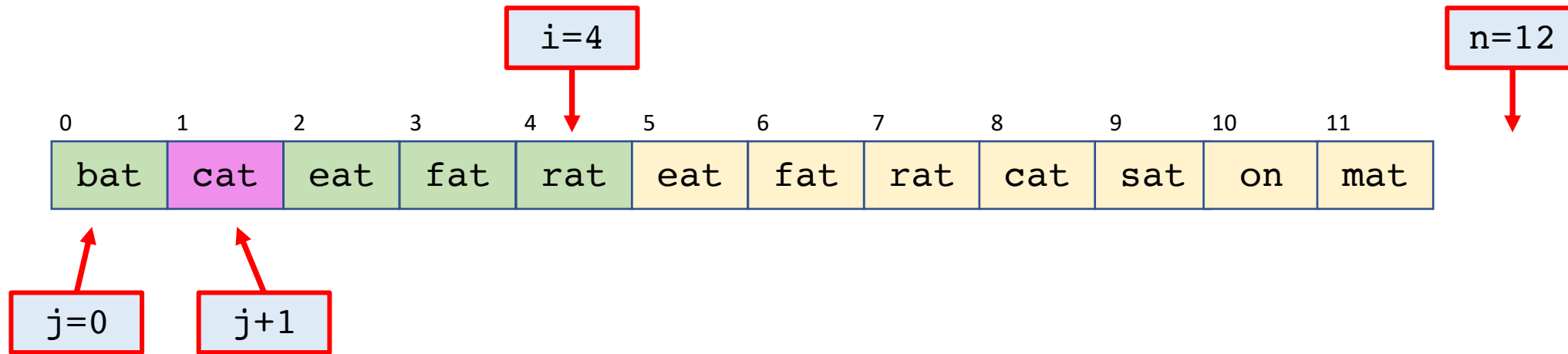
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



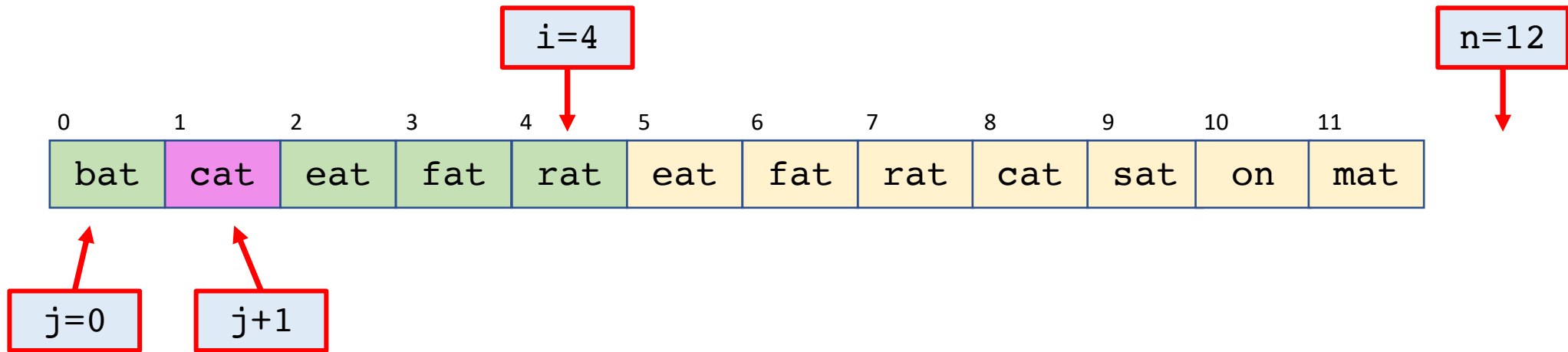
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

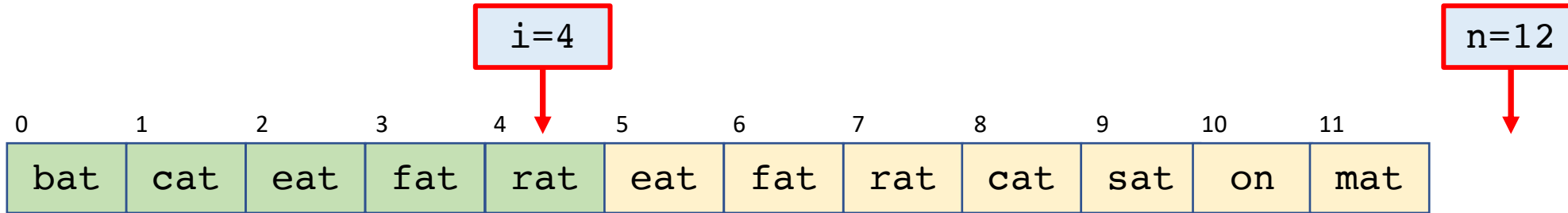
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct p */
    for (j=i-1; j>=0 && A[j+1]<A[j]; /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

desired **invariant**:
the elements to the left of **i** are sorted

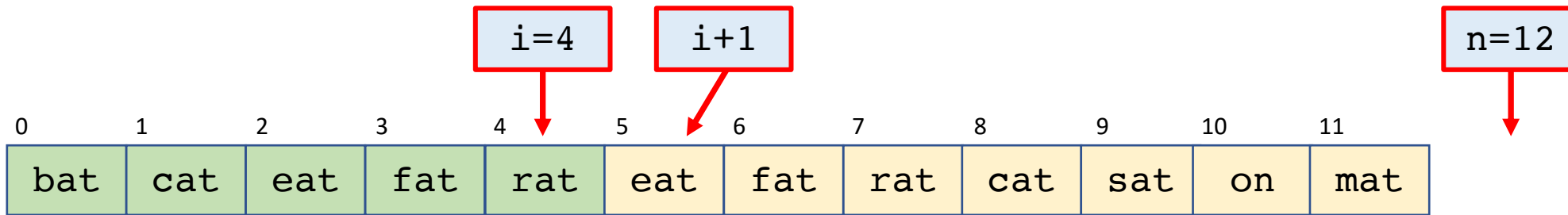
desired invariant:
the elements to the left of i are sorted

inductive step:
when the elements to the left of $i+1$ are sorted...

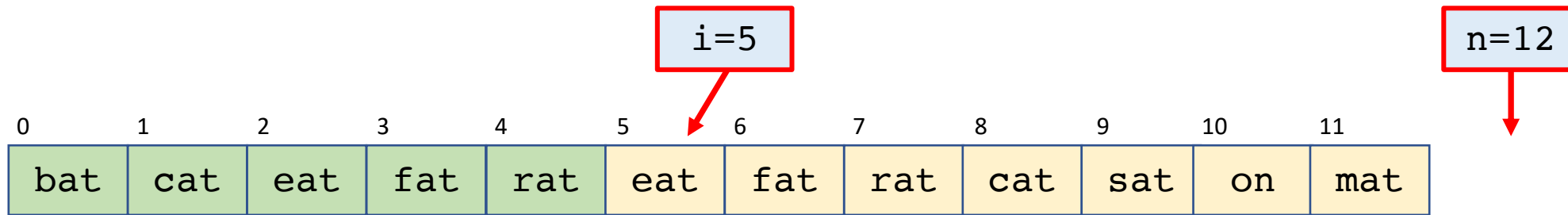
desired invariant:
the elements to the left of i are sorted

inductive step:
when the elements to the left of $i+1$ are sorted...

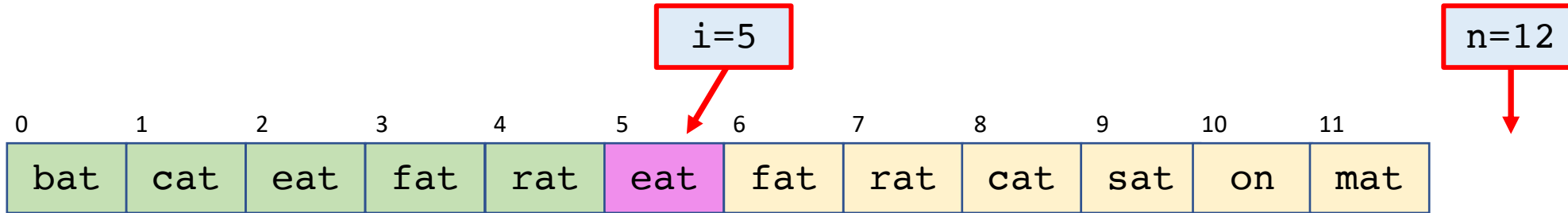
the assignment statement
 $i=i+1$
cannot violate the invariant



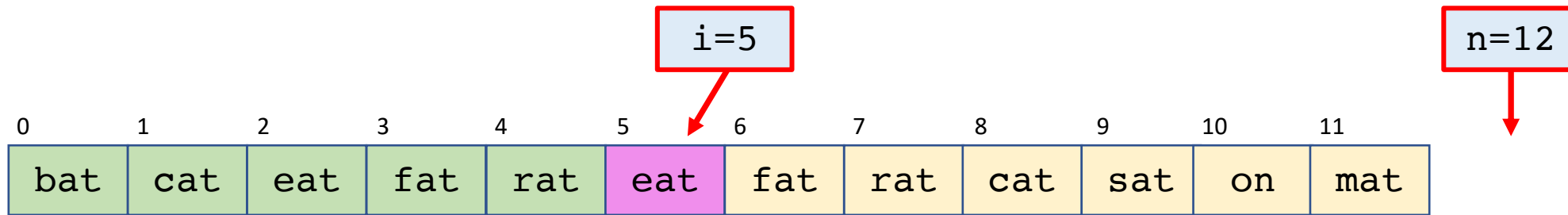
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

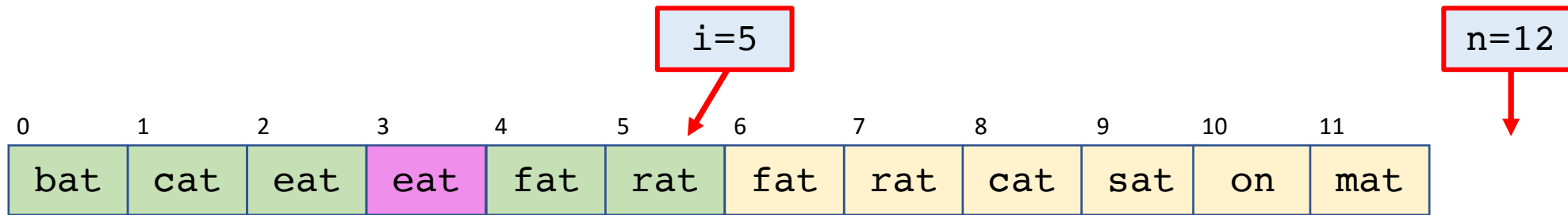
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



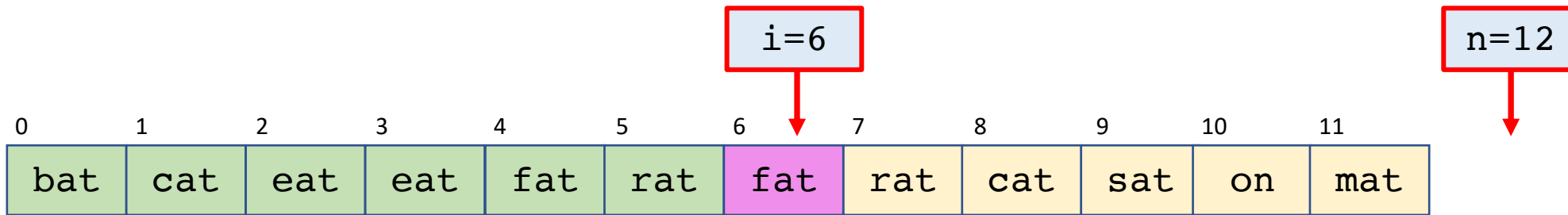
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



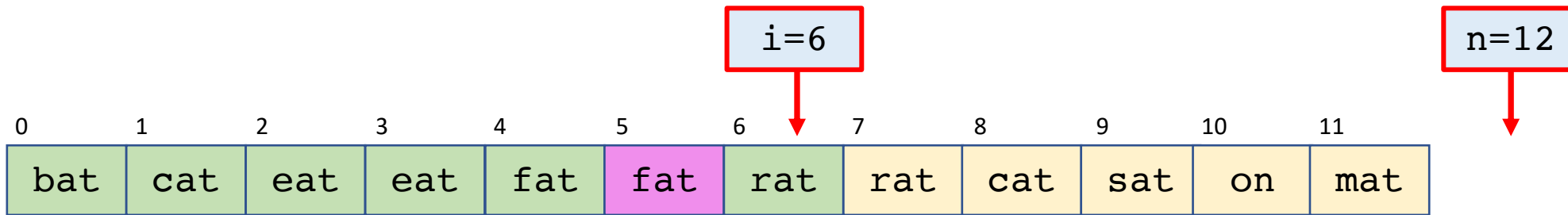
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



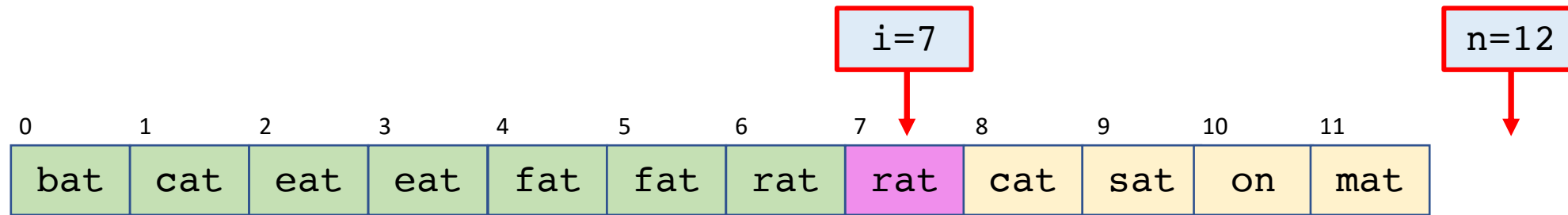
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



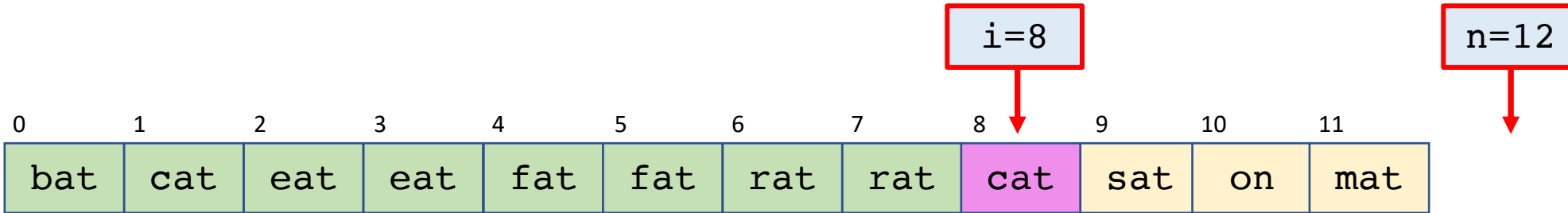
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



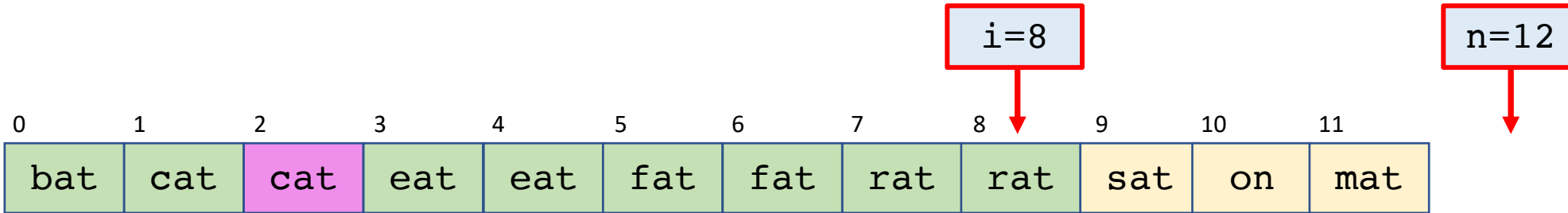
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



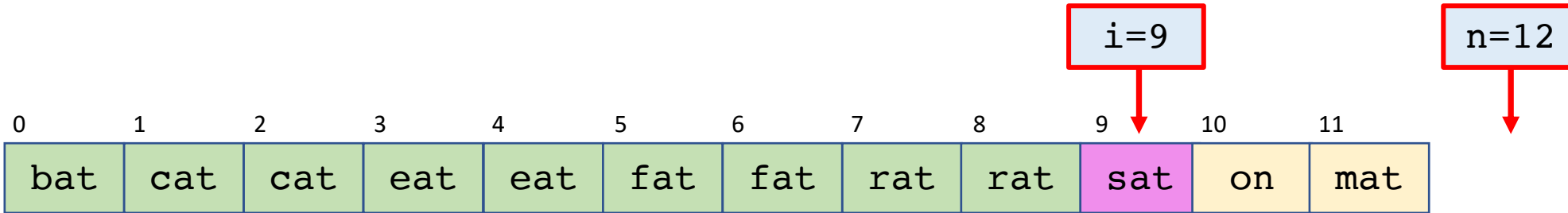
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



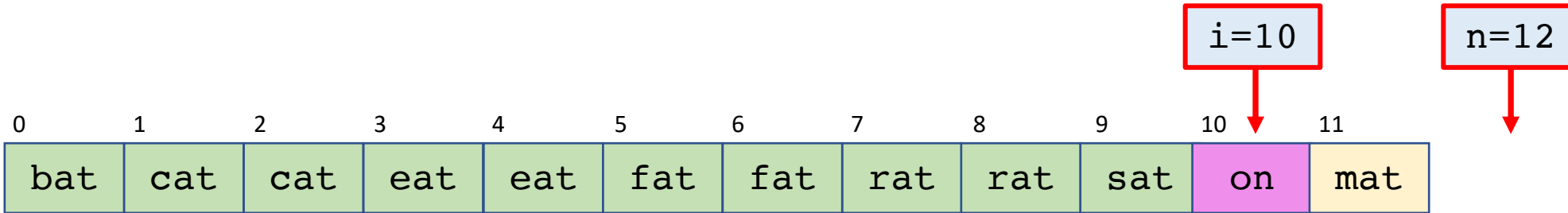
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

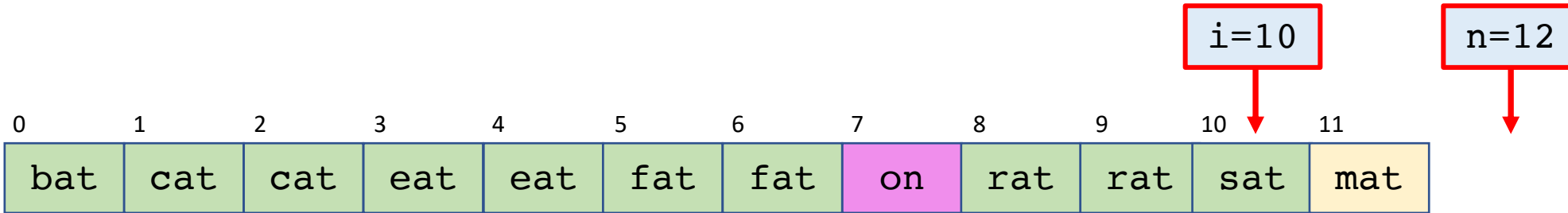
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



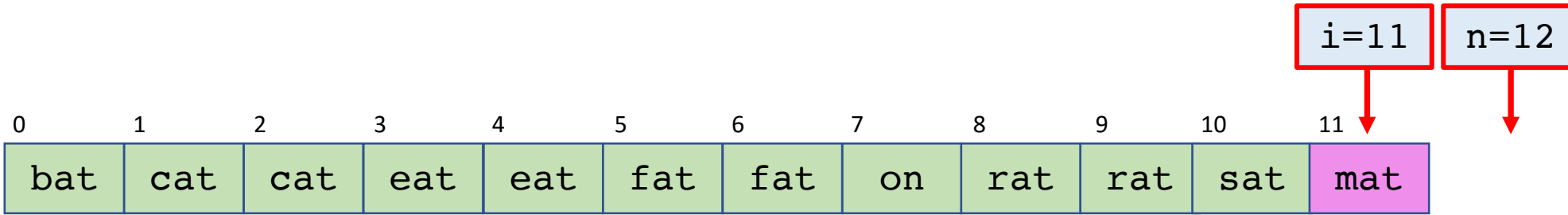
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



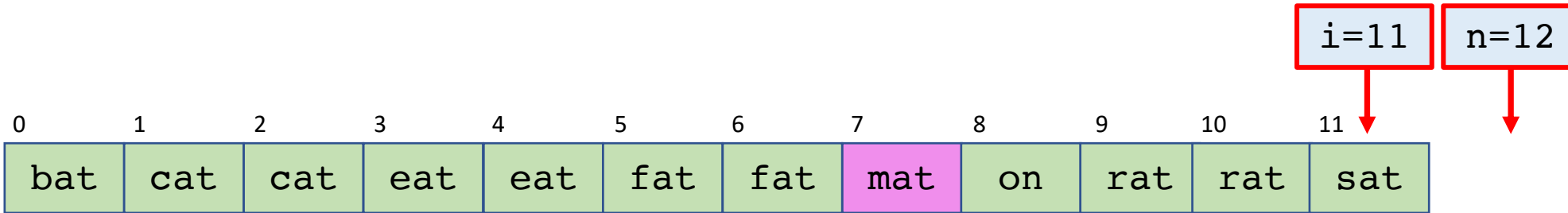
```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

0	1	2	3	4	5	6	7	8	9	10	11
bat	cat	cat	eat	eat	fat	fat	mat	on	rat	rat	sat

n=12



i=12



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

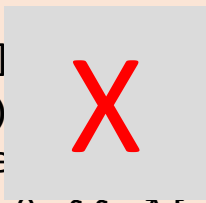
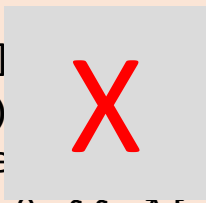
0	1	2	3	4	5	6	7	8	9	10	11
bat	cat	cat	eat	eat	fat	fat	mat	on	rat	rat	sat

n=12



i=12



```
/* assume that A[0]  -1] have valid values */  
for (i=1; i<n; i++)  
    /* swap A[i] le  correct position */  
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {  
        /* not there yet */  
        swap(&A[j], &A[j+1]);  
    }  
}  
/* and that's all there is to it! */
```


0	1	2	3	4	5	6	7	8	9	10	11
bat	cat	cat	eat	eat	fat	fat	mat	on	rat	rat	sat

n=12



i=12



```
/* assume that A[0] to A[n-1] have valid values */
for (i=1; i<n; i++) {
    /* swap A[i] left into correct position */
    for (j=i-1; j>=0 && A[j+1]<A[j]; j--) {
        /* not there yet */
        swap(&A[j], &A[j+1]);
    }
}
/* and that's all there is to it! */
```

demonstrated invariant:
the elements to the left of **i** are sorted...

demonstrated invariant:
the elements to the left of **i** are sorted...

and after the outer loop exits,
i==n...

demonstrated invariant:
the elements to the left of **i** are sorted...

and after the outer loop exits,
i==n...

therefore,
A[0] to **A[n-1]** are sorted!

what about the running time?

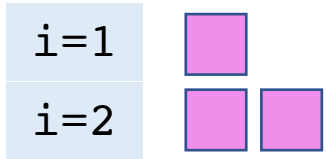
$i=1$



1

worst case:

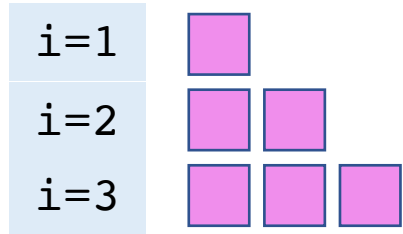
when n elements to be sorted,
 i ranges from 1 to $n-1$



$$1 + 2 = 3$$

worst case:

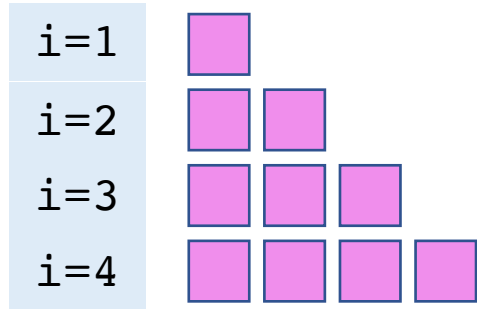
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6

worst case:

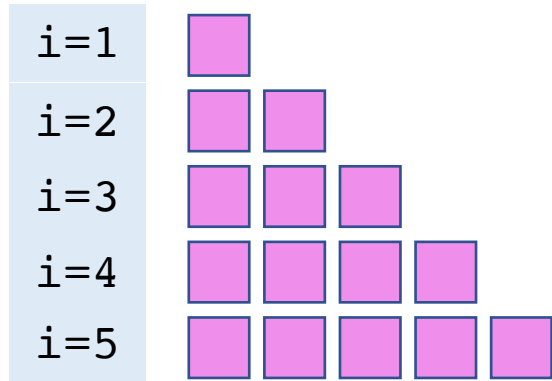
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10

worst case:

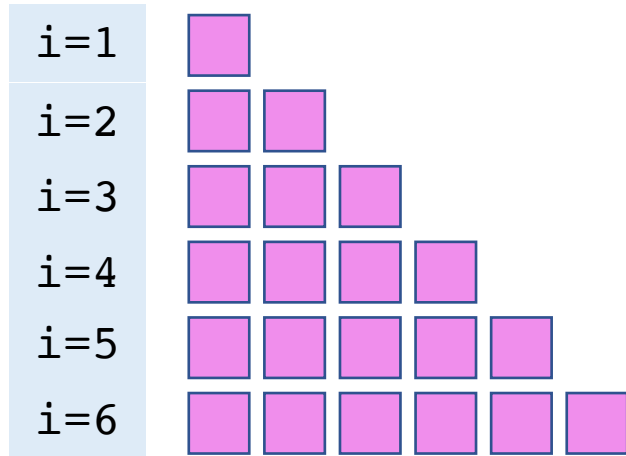
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15

worst case:

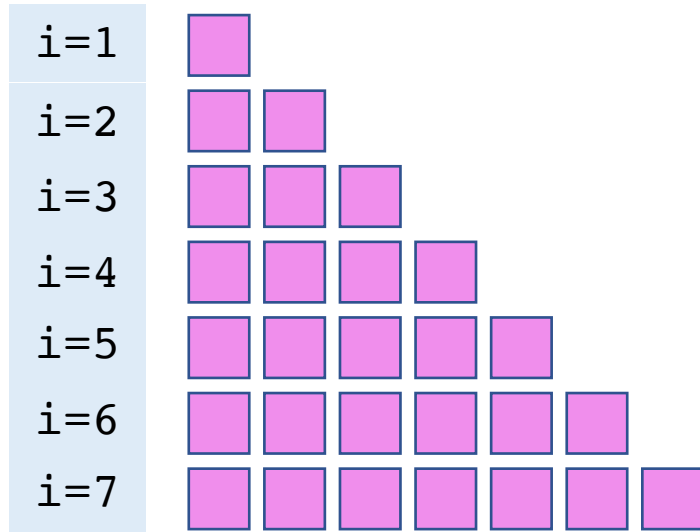
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21

worst case:

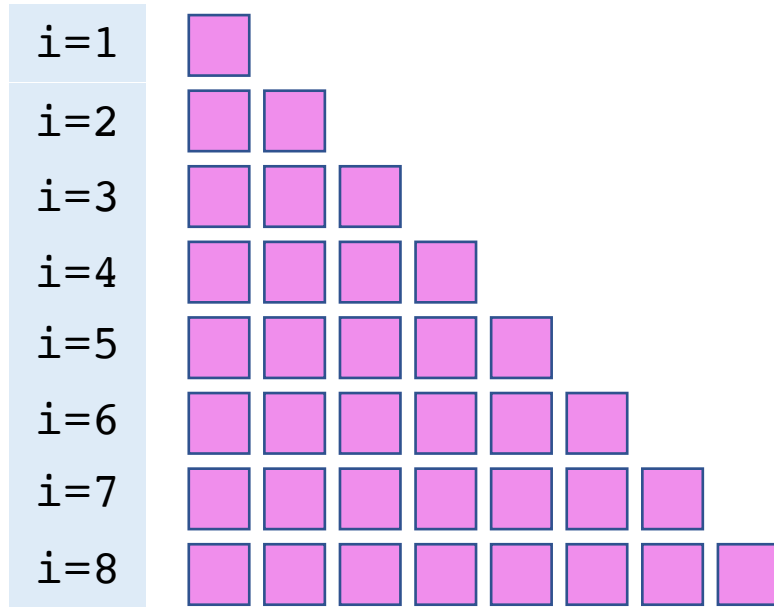
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28

worst case:

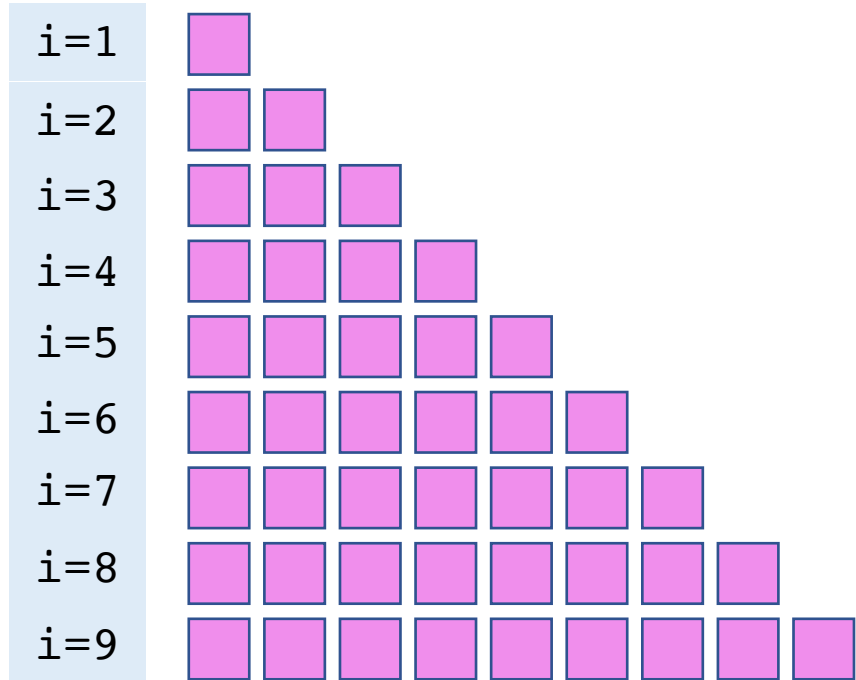
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28
+8=36

worst case:

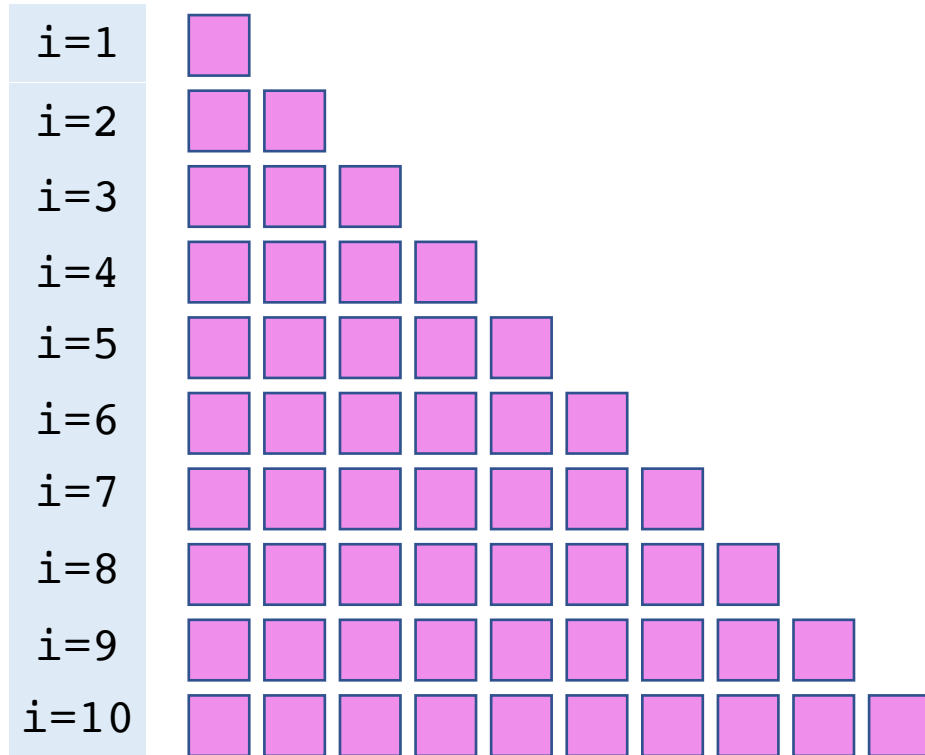
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28
+8=36
+9=45

worst case:

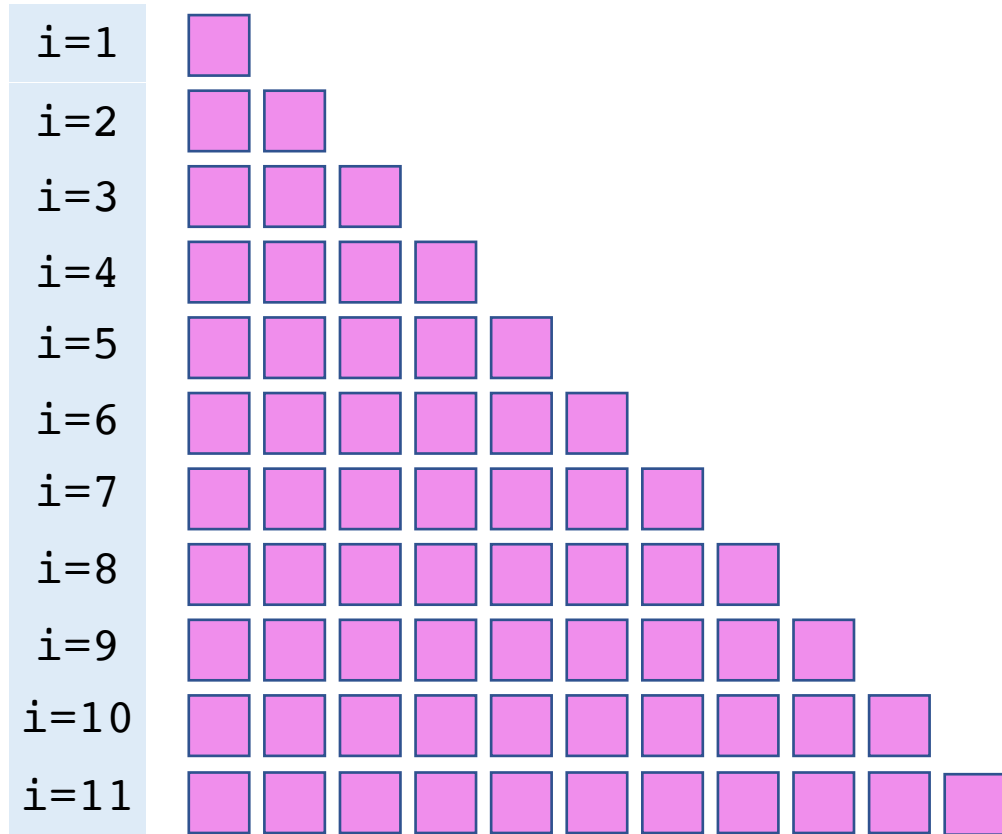
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28
+8=36
+9=45
+10=55

worst case:

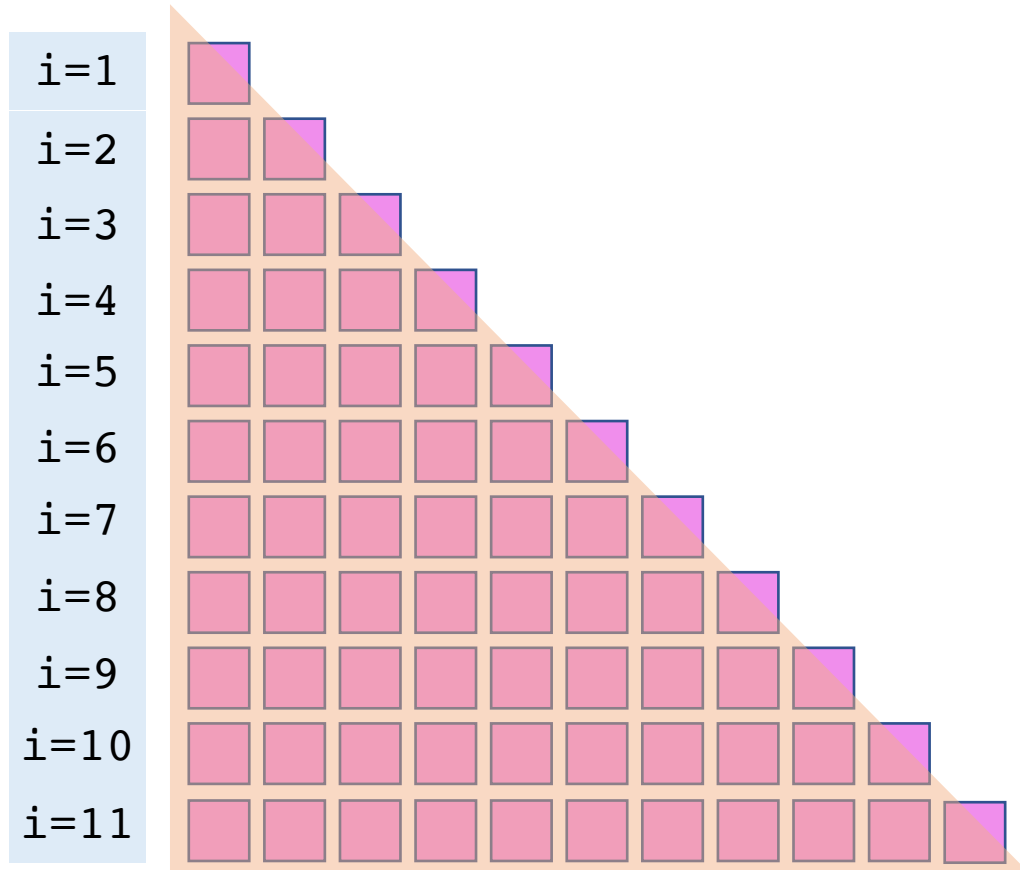
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28
+8=36
+9=45
+10=55
+11=66

worst case:

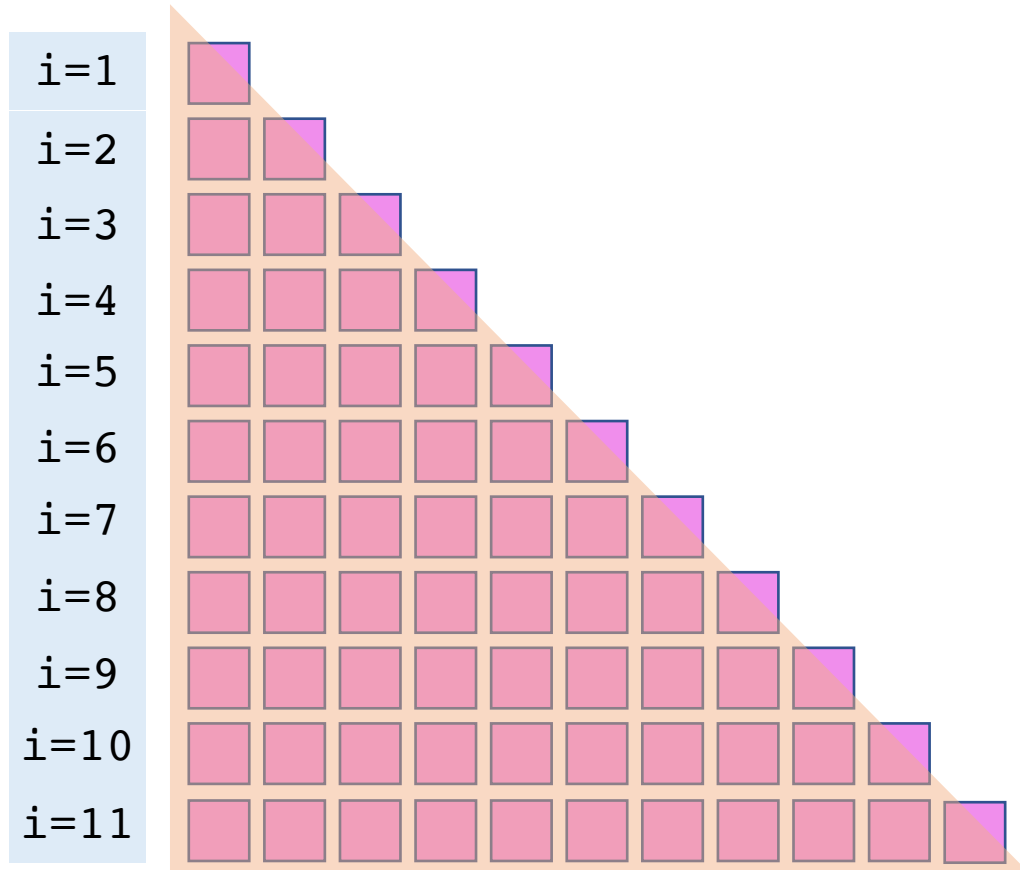
when n elements to be sorted,
 i ranges from 1 to $n-1$



1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28
+8=36
+9=45
+10=55
+11=66

worst case:

when n elements to be sorted,
 i ranges from 1 to $n-1$

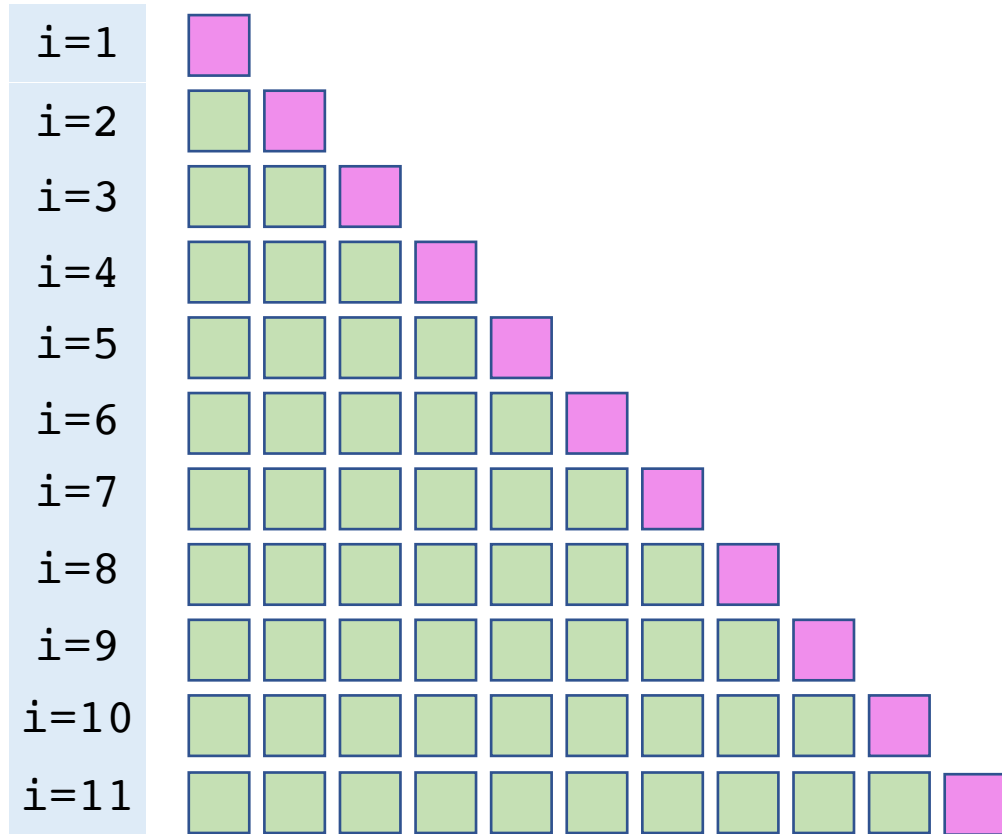


1
+2=3
+3=6
+4=10
+5=15
+6=21
+7=28
+8=36
+9=45
+10=55
+11=66

worst case:

when n elements to be sorted,
 i ranges from 1 to $n-1$

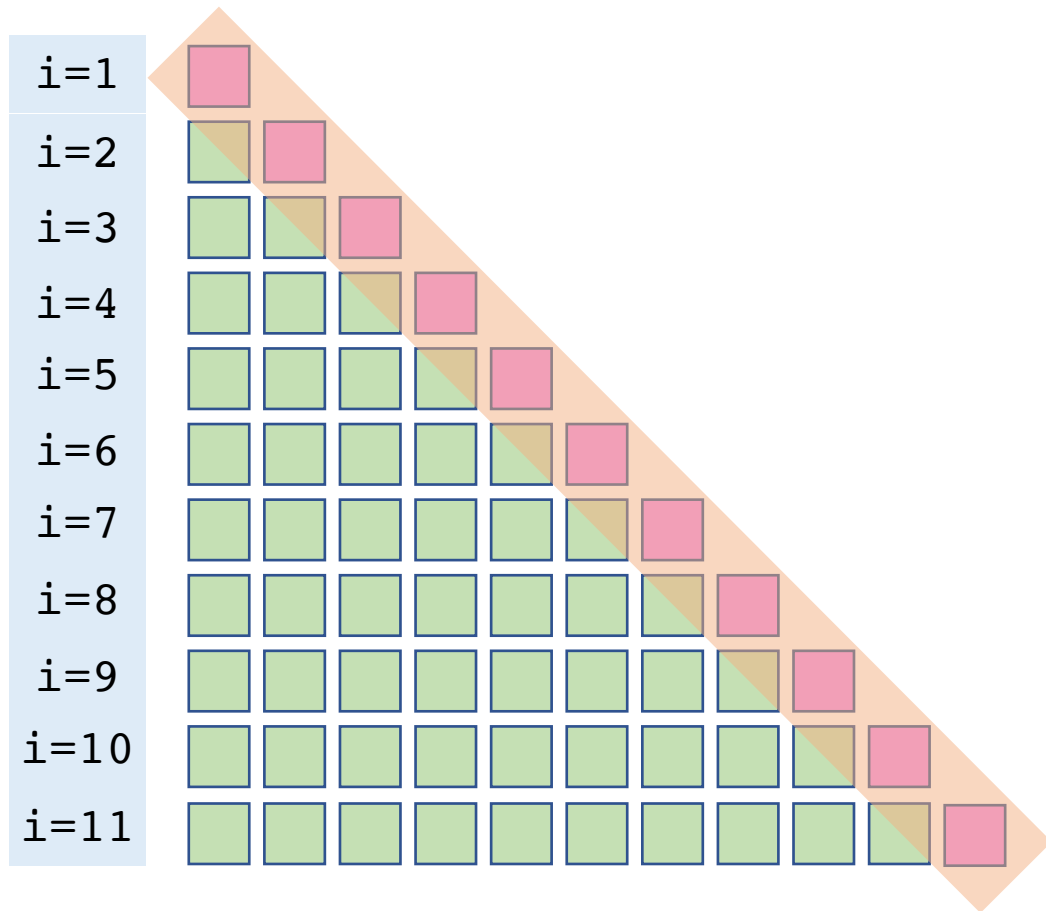
and as many as
 $n * (n-1) / 2$
comparisons might be
required



1
+1=2
+1=3
+1=4
+1=5
+1=6
+1=7
+1=8
+1=9
+1=10
+1=11

best case:

when n elements to be sorted,
 i ranges from 1 to $n-1$

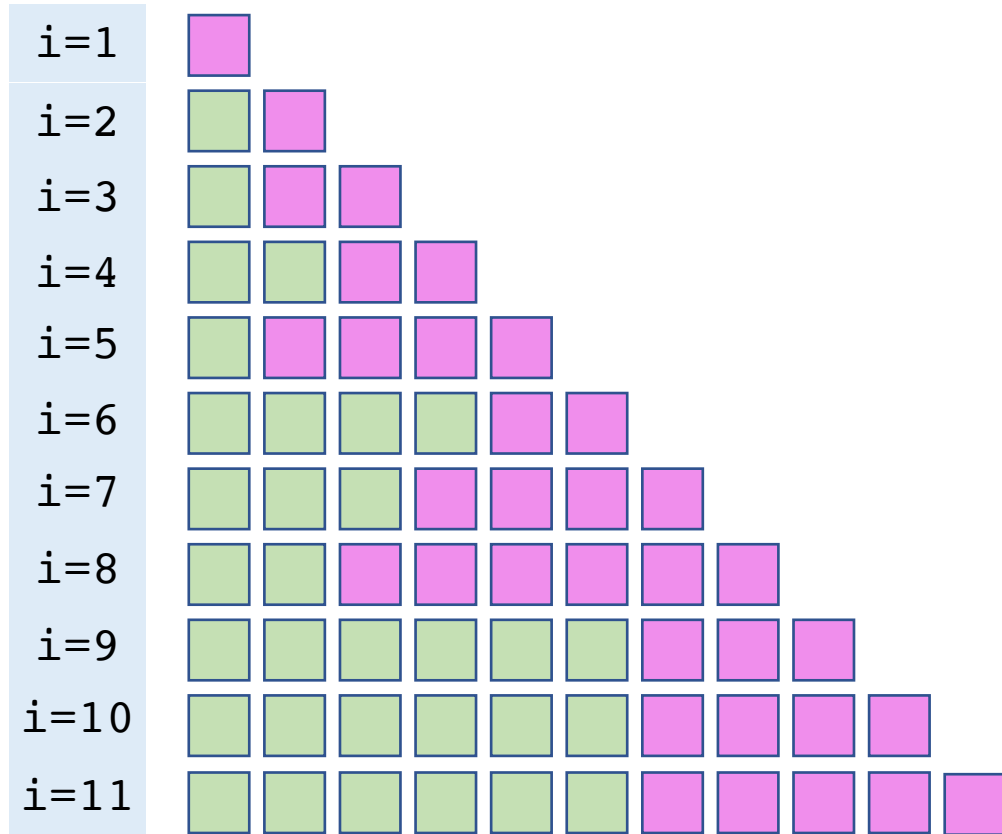


1
+1=2
+1=3
+1=4
+1=5
+1=6
+1=7
+1=8
+1=9
+1=10
+1=11

best case:

when n elements to be sorted,
 i ranges from 1 to $n-1$

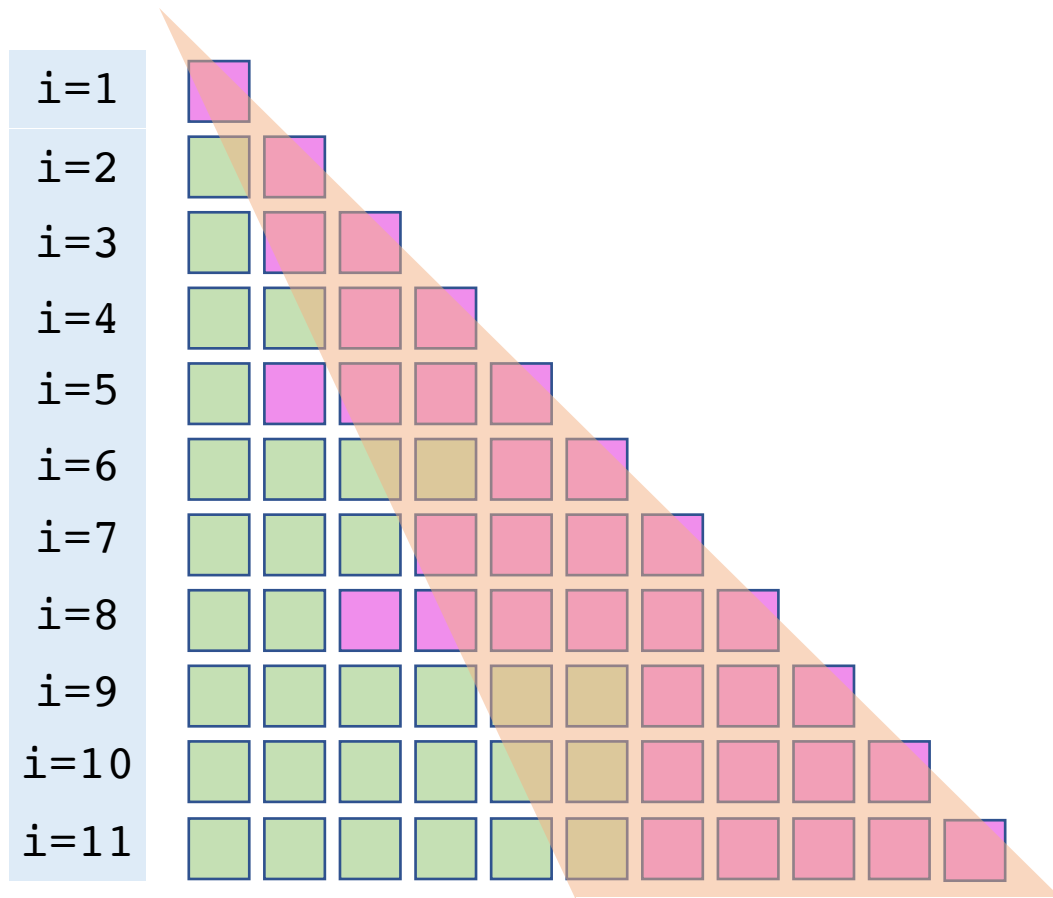
as few as
 $n-1$
comparisons might be
sufficient



1
+1=2
+2=4
+2=6
+4=10
+2=12
+4=16
+6=22
+3=25
+4=29
+5=34

average case:

when n elements to be sorted,
 i ranges from 1 to $n-1$



1
 +1=2
 +2=4
 +2=6
 +4=10
 +2=12
 +4=16
 +6=22
 +3=25
 +4=29
 +5=34

average case:

when n elements to be sorted,
 i ranges from 1 to $n-1$

around

$$n * (n-1) / 4$$
 comparisons will be required